Algebraic enumeration of generalized stack permutations

Columbus Labs
info@columbus-labs.com

12 October 2026

© 2026 Columbus Labs. Licensed under CC BY 4.0.

Abstract

A generalized (r, s)-stack inserts each successive input entry into one of its top r positions and removes an entry from one of its top s positions. We prove that, for every fixed pair of positive integers r, s, the ordinary generating function of the permutations sortable by this machine is algebraic. This answers Question 5.12 of Vatter's 2026 problem list. The proof counts each permutation through its unique lazy endpoint diagram. An exact endpoint-peeling criterion is monadic second-order definable, and a geometric argument bounds the diagram's treewidth by $12\max(r, s) - 1$. Established decomposition transductions, together with an explicit canonical selection and finite-port encoding, give a size-preserving bijection to a regular tree language. Separately, for $(r, s) = (3, 3)$ we give a 69-term polynomial of degree 12 in the generating function and degree 6 in the size variable, specify its counting branch, and prove its irreducibility over $\mathbb{Q}(x)$. A Lean certificate identifies that branch with the all-length count of distinct outputs of the literal unrestricted-height machine and proves the scalar relation. The general algebraicity argument and the degree-minimality argument are written proofs outside that formal certificate.

Keywords. Generalized stacks; permutation patterns; algebraic generating functions; monadic second-order logic; treewidth; formal verification.

1 Introduction

A single generalized stack, introduced by Atkinson [3], permits bounded access near the top while allowing unbounded storage height. With push width r and pop width s, each operation moves one entry. The parameters specify accessible positions, not the sizes of contiguous blocks. In particular this is a different model from a forkstack [1], from two stacks in series, and from iteration of the deterministic stack-sorting map.

Let $\mathcal{G}(r, s)$ be the permutations generated from increasing input, and let $\mathcal{S}(r, s)$ be the permutations that can be sorted into increasing order. Write

$$F_{r,s}(x) = \sum_{n \geq 0} g_{r,s}(n) x^n, \qquad g_{r,s}(n) = |\mathcal{G}(r, s) \cap \mathfrak{S}_n|.$$

The empty permutation is included once. Relabelling a sorting history by input positions shows that the two classes are inverse to each other, so this is also the ordinary generating function of $\mathcal{S}(r, s)$.

Vatter [18], Question 5.12, asks whether these series are algebraic for all positive widths. We prove the following.

Theorem 1.1. *For every fixed pair of positive integers r, s, the series $F_{r,s}(x)$ is algebraic over $\mathbb{Q}(x)$. Both input length and stack height are unrestricted.*

The main difficulty is avoiding multiplicity. A physical machine may realize a permutation by several histories; a graph may have several bounded-width decompositions; and a nondeterministic automaton may have several accepting runs. Counting any of these witnesses generally gives the wrong sequence. We remove the three multiplicities separately: lazy schedules encode permutations uniquely, a definable order canonically selects the decomposition data, and a deterministic tree automaton counts the resulting encodings.

The first part of the proof is stack-specific. The lazy diagram has two event vertices for each token, a directed time path, and a perfect insertion–removal matching. Its feasibility is expressed by requiring an eligible bottom token in every nonempty token subsystem. A drawing by nested or crossing U-shaped curves bounds the treewidth of the sparse path-plus-matching graph. The dense time order is defined by reachability; it is never added to this graph.

The second part uses the definable-decomposition machinery of Bojańczyk and Pilipczuk [6, 7], in the original-vertex introduction-order formulation of Bojańczyk's lecture notes [5, Definitions 6.20–6.21 and Theorem 6.22]. We construct an elimination tree on those vertices, select a least tuple of monadic choices, and encode the result with finitely many ports. Exact bags and ordered children remove the remaining choices. This produces a regular language with one tree node per event. Graph parsability, from Courcelle's strongly context-free graph languages [9] to Iosif and Zuleger [13], is the underlying precedent. The counting argument must additionally establish a size-preserving one-to-one correspondence.

1.1 A concrete equation at width three

For $G(x) = F_{3,3}(x)$, a separate finite counting construction gives substantially more than the general existence theorem.

Theorem 1.2. *Let $Q(x, y)$ be the integer polynomial in Appendix A. It has 69 nonzero terms, degree 12 in y, and degree 6 in x. Then*

$$Q(x, G(x)) = 0,$$

and G is its unique formal branch with prefix

$$\begin{aligned} G(x) = 1 + x + 2x^2 + 6x^3 + 24x^4 + 120x^5 + 720x^6 \\ + 5004x^7 + 38844x^8 + 325692x^9 + O(x^{10}). \end{aligned}$$

Moreover, Q is irreducible over $\mathbb{Q}(x)$, so G has algebraic degree exactly 12.

The general theorem is independent of this concrete construction. The latter uses a signed completion recurrence, an exact finite invariant representation, and rational reconstruction from a nonsingular scalar branch. Section 7 gives the proof chain and identifies its finite exact certificates. The expanded algebraic identities and the large kernel-checked proof are supplementary data, rather than hidden finite-prefix assumptions.

1.2 Related work and scope

Atkinson established the machine model and enumerated its one-sided cases [3]. The nearby algebraicity theorem for forkstacks [1] concerns contiguous block transfers, so it does not settle

the top-position question. Restricted-container machines [2] supply other algebraicity criteria, in a model whose ordinary output operation removes the leftmost entry. Those criteria also do not directly cover all top-s removals.

There are also direct precedents for using logic to enumerate permutation classes. Braunfeld [8, Section 4 and Theorem 5.2] constructs size-preserving regular-word encodings for geometric grid classes and their MSO-defined subclasses, using definable least representatives to remove ambiguity. Jelínek and Opler [14, Theorem 4.1] study MSO model checking parameterized by the treewidth of a permutation's two-successor graph. That graph differs from our endpoint time-path-plus-matching graph. These results inform the method; they do not state algebraicity for all of Atkinson's top-position stacks.

Theorem 1.1 is an existence theorem for a finite unambiguous polynomial system. It does not claim a practical general automaton, an efficient coefficient algorithm obtained from that automaton, or an asymptotic formula. No parameter is allowed to increase with n. We make no claim to have settled stack sorting in general, and no claim that the logical decomposition machinery is new. A bounded primary-source comparison found no exact predecessor resolving the stated top-position question; this is not a proof of historical priority.

2 The literal machine and its unique lazy diagram

Write a stack top first. At height h, a push inserts the next unread input entry into a slot

$$0 \le j \le \min(r-1, h),$$

and a pop removes the entry in a slot

$$0 \le j \le \min(s-1, h-1).$$

A complete history starts and ends with an empty stack. There is no bypass operation. An entry that is pushed and immediately popped occupies physical stack space between these two operations.

Lemma 2.1 (Inversion)**.** *A permutation π belongs to $\mathcal{S}(r,s)$ if and only if π^{-1} belongs to $\mathcal{G}(r,s)$.*

Proof. Relabel every entry in a sorting history by its position in the input word. The input becomes $1,\dots,n$, while the output becomes the list of positions of $1,\dots,n$ in π, namely π^{-1}. Slot choices are unchanged. The relabelling is reversible. □

Lemma 2.2 (Lazy normalization)**.** *If increasing input generates π, it does so by a history that reads new input only when the next required output is a new left-to-right maximum, and then reads exactly through that maximum before removing it. This normalization does not increase physical capacity.*

Proof. Suppose a push of a is immediately followed by a pop of a different token b. Write h for the height before the push, j for its insertion slot, and i for b's slot after it. The token b was already present, so $i \neq j$. Commute the pop to the left. If $i < j$, use pop slot i and then push slot $j-1$; here $i \le h-1$ and $0 \le j-1 \le h-1$. If $i > j$, use pop slot $i-1$ and then push slot j; here $0 \le i-1 \le h-1$ and $j \le i-1 \le h-1$. In both cases the new pop slot is no larger than $i < s$, and the new push slot is no larger than $j < r$. Thus both the width bounds and the changed height caps hold. The final stack is identical, while the intermediate height decreases.

Normalize output boundaries from left to right. Between two pops, the operations form a run of pushes. If the next output is held already, move the entire run after that output. If it is a new

maximum v, retain exactly the pushes through v and postpone all later pushes. The commutations preserve input order and do not alter earlier output boundaries. Induction gives the required history. □

A *schedule* is a finite word of insertion and removal events with token identities, each token inserted once and later removed once. Its input order is the order of insertions. It is *realizable* if legal slot choices produce exactly these events. Erasing both events of selected tokens gives a restricted schedule, which need not be lazy.

The normalized sequence of token insertions and removals is determined by π, even though physical slot choices need not be unique. At a new record v, insert the unread labels through v and then remove v; at any other output, remove the requested held label. We call this the *lazy schedule*.

Definition 2.3. The *endpoint diagram* of any nonempty schedule has one vertex for every insertion or removal event. It carries unary insertion and removal colors, the directed successor relation along time, and a symmetric matching joining the two events of each token. Its underlying simple graph D has precisely the undirected time-adjacency and matching edges.

Lemma 2.4. *Permutations of length $n \geq 1$ are in bijection with isomorphism classes of diagrams having the following properties:*

(i) the successor relation is a directed path through every vertex;
(ii) insertion and removal colors partition the vertices;
(iii) the matching is perfect, joins opposite colors, and every insertion precedes its matching removal;
(iv) whenever an insertion is immediately followed by a removal, those events are matched.

Under the bijection, a permutation is generated by the (r, s)-stack exactly when its diagram is physically realizable with those widths.

Proof. A lazy schedule has the stated properties. Conversely, label insertion events $1, \ldots, n$ in time order and output their matched labels at the removal events. Perfect matching ensures that this is a permutation. The final event cannot be an insertion, by (iii). Thus every maximal insertion run is followed by a removal, and (iv) identifies that removal with the final insertion in the run. Its label is therefore a new output maximum. Since insertion labels are assigned in time order, this run reads exactly the unread labels through that maximum. Between such runs the removals output held labels. Hence the recovered schedule is exactly the lazy schedule of the recovered permutation. These constructions are inverse. A path-preserving isomorphism preserves every event position, so there is no rotational, reversal, or labeling multiplicity. The final statement follows from Lemma 2.2. □

3 Endpoint peeling characterizes feasibility

For a token v, let $a_v < b_v$ be its insertion and removal event indices. The token is held in the open interval (a_v, b_v). For a set U of tokens and $v \in U$, put

$$A_U(v) = \{u \in U : a_u < a_v < b_u\}, \qquad B_U(v) = \{u \in U : a_u < b_v < b_u\}.$$

Strict inequalities exclude the token itself. Call v *eligible in U* if $|A_U(v)| < r$ and $|B_U(v)| < s$.

Lemma 3.1 (Global physical order)**.** *Every realizable schedule has a total order on all its tokens whose restriction to the held tokens at each time is the physical stack order.*

Proof. Orient every edge of the interval intersection graph from the physically upper token to the lower one. The orientation is well-defined because two simultaneously held tokens never exchange order. Three pairwise intersecting intervals have a common open time gap, by the interval Helly property and the distinctness of endpoints. The three tokens coexist there, so the orientation has no directed triangle.

The interval intersection graph is chordal. Indeed, in a putative chordless cycle of length at least four, choose the interval with earliest right endpoint. Its two cycle neighbors both contain that endpoint and intersect, giving a chord. A shortest directed cycle of length at least four would therefore have a chord, and either orientation of that chord would produce a shorter directed cycle. There are no directed cycles. Any total extension of this acyclic orientation has the required restrictions. □

Conversely, a total order realizes a schedule if its restrictions put each inserted token at depth below r and each removed token at depth below s. Insertions and removals at those restricted positions construct the history.

Lemma 3.2 (Bottom deletion)**.** *If v is eligible in U, the schedule on U is realizable if and only if the schedule obtained by erasing v is realizable.*

Proof. Erasing a token from a physical history only decreases the other operation depths. At every surviving event the restricted position is still a slot of the current restricted stack, so the height-dependent caps also hold. For the reverse direction, use Lemma 3.1 to take a realizing total order on $U \setminus \{v\}$ and place v below every other token. Existing endpoint depths do not change. The insertion and removal depths of v are exactly $|A_U(v)|$ and $|B_U(v)|$, which are legal by eligibility and the current height caps. The converse observation following Lemma 3.1 constructs the history. □

Proposition 3.3 (Every-subset criterion)**.** *A schedule is realizable by the (r, s)-stack if and only if every nonempty token subset U contains an eligible token.*

Proof. A restriction of a realizing history is realizable. The bottom token of a realizing total order on a nonempty restriction is eligible, proving necessity. Conversely, repeatedly choose an eligible token in the current residual set. The hypothesis applies at each step, and the finite process ends at the empty schedule. Restore tokens in reverse order using Lemma 3.2. □

The quantifier over *every* subset is essential. An initially eligible token alone does not certify the schedule. The total order used in the proof is an existence witness, not part of the counted diagram.

4 A uniform treewidth bound

Set $k = \max(r, s)$. The following geometric bound is deliberately conservative.

Theorem 4.1. *For every realizable (r, s) schedule, its endpoint diagram satisfies*

$$\mathrm{tw}(D) \leq 12k - 1.$$

Proof. Choose a global physical order as in Lemma 3.1. Give each token v a distinct positive integer depth y_v, increasing down this order. Place its two endpoints at $(a_v, 0)$ and $(b_v, 0)$, and draw the U-shaped curve

$$(a_v, 0) \longrightarrow (a_v, -y_v) \longrightarrow (b_v, -y_v) \longrightarrow (b_v, 0).$$

Distinct event coordinates and depths ensure that every crossing is a transverse intersection of one vertical stem with one horizontal segment. There are no triple crossings, coincident segments, or crossings at bends. The bends are not graph vertices.

A vertical stem of v crosses exactly the horizontal segments of tokens that are active at that endpoint and physically above v. The insertion stem has at most $r-1$ crossings; the removal stem has at most $s-1$. Thus each stem has at most $k-1$ crossings. A horizontal segment may have arbitrarily many crossings, and no bound on that number is needed.

Add edges between consecutive event endpoints. If there are at least two tokens, join the first and last endpoints by a curve around and below all U-curves, disjoint from the drawing except at its ends. All event endpoints remain incident with the unbounded upper face. Replace crossings by degree-four vertices and suppress geometric bends to obtain a plane multigraph P; crossing lenses may create parallel edges. The one-token case is immediate.

Every crossing lies on a vertical stem and has graph distance at most $k-1$ in P to an outer-face endpoint. In each peeling round, delete all vertices on the current outer face. Every surviving neighbor of a deleted vertex is on the new outer face: its incident edge opens into the old outer face. Induction along a shortest path therefore deletes any vertex at distance at most d from an original outer-face vertex within $d+1$ rounds, also for disconnected residual drawings. Thus P is k-outerplanar. Deleting parallel edges can only enlarge faces and cannot increase the number of peeling rounds, so its underlying simple graph is also k-outerplanar.

The standard outerplanarity bound gives $\mathrm{tw}(P) \le 3k-1$ [4]; we use the statement and proof in [15, Theorem 3.8]. Take a decomposition of P with bags of size at most $3k$. At an original endpoint, record its one token; at a crossing, record both crossing tokens. Replace each bag vertex by *both endpoints of every token recorded there.* Each old vertex contributes at most four new vertices, so the new bags have size at most $12k$.

These bags form a decomposition of the endpoint graph, with the optional closing edge. A matching edge is contained in every lifted bag meeting its token curve. Each time edge is covered because its endpoints already share a bag of P. Finally, a token curve is a connected path in the planarization. Bags meeting successive path vertices have intersecting occurrence subtrees because a bag covers their common edge. Their union is connected. This union is exactly the set of lifted bags containing either original endpoint of that token, proving the running-intersection property. Deleting the optional closing edge preserves the bound. □

Remark 4.2. The geometric layout and the decomposition are not counted decorations. They prove a width bound for the sparse time-path-plus-matching graph. The result says nothing comparable about adding all pairs of the time order, nor about replacing this graph by the usual numerical/output adjacency graph of a permutation.

5 A canonical regular-tree encoding

We state the logical bridge with the hypotheses used in the application. Monadic second-order logic (MSO) allows quantification over individual elements and sets of elements. Edge-set quantification is understood via the incidence representation of the underlying simple graph.

Theorem 5.1 (Ordered bounded-width encoding)**.** *Fix a finite vocabulary of unary relations and loopless binary relations, a nonnegative integer W, a parameter-free MSO formula $<$, and an MSO sentence φ. Let $\mathcal{A}$ be an isomorphism-closed class of nonempty finite connected structures defined*

by φ, all of whose underlying simple graphs have treewidth at most W and on each of which $<$ defines a strict linear vertex order. Thus $\mathcal{A}$ is exactly the class defined by this sentence in the intended vocabulary, not an arbitrary subclass of its models. Then their isomorphism classes have a size-preserving bijection to a regular language of finite ranked trees over a finite alphabet, with one tree node per original vertex. Their ordinary generating function by vertices is algebraic.

The underlying graph forgets directions and relation colors. Several binary relations may hold on the same pair. Loops are excluded so that no diagonal relation data are omitted by the encoding. The order must be definable from the structure being counted; an arbitrary extra order forgotten after counting does not satisfy the theorem's counting specification.

5.1 Vertex-preserving elimination trees

For a rooted forest F on the vertices of a graph, define

$$B_F(v) = \{v\} \cup \{u : u \text{ is a strict ancestor of } v \text{ and has a neighbor in the } F\text{-subtree of } v\}. \quad (1)$$

An elimination forest has ancestor-comparable endpoints for every graph edge. Its width is $\max_v |B_F(v)| - 1$.

We require formulas on the original vertices, not merely an isomorphism-invariant relation between inputs and outputs of a transduction. This input is stated directly in Bojańczyk's lecture notes [5, Definitions 6.20–6.21 and Theorem 6.22], which present the definable-decomposition result of [6]. For fixed W, there are a constant $K = K(W)$ and an MSO formula $\rho(\mathbf{X}, u, v)$ with a fixed finite tuple of set parameters on the original incidence universe, such that every graph of treewidth at most W admits parameters defining the *introduction preorder* of a tree decomposition of width at most K. This preorder says that the node introducing u is equal to or an ancestor of the node introducing v. No optimal-width or origin-preservation assertion about a composed transduction is needed.

Here is the explicit conversion to a forest on the original vertices. In the following formulas, u, v, z range over the vertex sort; the sets $\mathbf{X}$ may use the full incidence universe. With parameters fixed, define

$$A_{\mathbf{X}}(u, v) := \rho(\mathbf{X}, u, v) \ \wedge \ (\neg\rho(\mathbf{X}, v, u) \ \vee \ u < v).$$

For parameters representing an introduction preorder, this replaces each class of simultaneously introduced vertices by its $<$-ordered chain. The resulting strict partial order has a linearly ordered predecessor set at each vertex. Its cover relation

$$P(\mathbf{X}, u, v) := A_{\mathbf{X}}(u, v) \ \wedge \ \neg\exists z \, (A_{\mathbf{X}}(u, z) \wedge A_{\mathbf{X}}(z, v))$$

is consequently a rooted-forest parent relation on exactly the original vertex set. The introduction nodes of the endpoints of a graph edge are comparable, since some decomposition bag contains both. Hence the forest is an elimination forest.

Its width is at most K. Let t_v be the decomposition node introducing v. If $u \in B_F(v) \setminus \{v\}$, then u is an ancestor of v in the new forest and is adjacent to some descendant w of v. The introduction nodes of u, v, w occur in that order along a decomposition-tree branch, allowing equality. A bag containing u, w is at or below the introduction node of w. Running intersection for u, from its introduction node to that bag, forces u into the bag at t_v. The vertex v is there as well. Thus $B_F(v)$ is a subset of the original bag at t_v, whose size is at most $K + 1$.

Define $\delta(\mathbf{X})$ to check that $P(\mathbf{X}, \cdot, \cdot)$ is a rooted forest on all original vertices, that every graph edge has ancestor-comparable endpoints, and that every bag in (1) has size at most $K+1$. These are MSO conditions: express ancestry by monadic reachability, require at most one parent and no directed cycle, and forbid $K+2$ distinct members of a bag. On finite vertex sets these conditions characterize the required forests. Therefore every width-at-most-W input has an admissible tuple by the preceding construction, and *every* admissible tuple has the required same-vertex meaning by the definition of δ. The fixed port bound below is K, which may exceed W; only finiteness is needed.

5.2 Definable canonical choice

Extend $<$ to the incidence universe by placing vertices first, in their order, and then simple edges in lexicographic order of their endpoint pairs $(\min_<, \max_<)$. The extension is MSO-definable. There is one incidence element per simple edge; overlapping edge-relation flags do not create extra elements.

Read the truth values of $X_1, \ldots, X_c$ in this fixed order, with $0 < 1$. This gives a lexicographic order on the finite set of possible tuples. The relation $\mathbf{Y} <_{\text{lex}} \mathbf{X}$ is MSO-definable: a finite disjunction specifies the first differing predicate at the first differing incidence element, and all preceding bits are required to agree. Define

$$\text{Canon}(\mathbf{X}) := \delta(\mathbf{X}) \ \wedge \ \neg\exists\mathbf{Y}\,(\delta(\mathbf{Y}) \wedge \mathbf{Y} <_{\text{lex}} \mathbf{X}).$$

There is exactly one admissible minimum. Consequently

$$P_{\text{can}}(u, v) := \exists\mathbf{X}\,(\text{Canon}(\mathbf{X}) \wedge P(\mathbf{X}, u, v))$$

defines a unique elimination forest on the original vertex set. Connectedness makes it a single tree, since no graph edge can join distinct forest components. The construction is invariant under isomorphisms, because the order is itself definable.

5.3 Finite ports and labels

Let F be the canonical tree. At a node v, port 0 names v itself. Order the other vertices of $B_F(v)$ by $<$ and give them ports $1, \ldots, d(v)$, with $d(v) \le K$. An inherited vertex in a child's bag occurs in the parent's bag: it is either that parent, or an earlier ancestor with a neighbor in the child's subtree, hence also in the parent's subtree. The child label can therefore record the injective map

$$f_v : \{1, \ldots, d(v)\} \longrightarrow \{0, \ldots, d(\text{parent}(v))\}.$$

At the root, $d(v) = 0$.

A label additionally records the node's unary colors and, for each inherited port, all binary relation/direction flags between that ancestor and v. This is finite information. An inherited port may have zero edge flags, since its relevant neighbor may lie deeper in the subtree. Every graph edge is stored exactly once, at its deeper endpoint. All flags on one underlying edge are retained together.

Order the children of each node by $<$ on their root vertices. Encode the ordered tree in first-child/next-sibling form. This creates no new nodes. A finite ranked alphabet of ranks $0, 1, 2$ includes tags distinguishing a lone first-child edge from a lone next-sibling edge; the root has no next sibling. Once the structure and its canonical parent relation are fixed, these labels and orders have no remaining choices.

5.4 MSO decoding and the same-node check

From the binary encoding recover the original parent relation by a first-child edge followed by zero or more next-sibling edges. Port identity propagates upward:

$$\mathrm{port}_0(v) = v, \qquad \mathrm{port}_i(v) = \mathrm{port}_{f_v(i)}(\mathrm{parent}(v)) \quad (i > 0).$$

Valid map bounds, injectivity, and the root condition ensure that every port reaches exactly one ancestor introduced at port 0, and that distinct ports at a node remain distinct. This is MSO-definable by reachability in the fixed finite-copy product of nodes with port states $0, \ldots, K$. A fixed tuple of monadic sets expresses that reachability; no unbounded binary matching is guessed.

The edge bits now decode the original relations. For incidence formulas, create one edge element (v, i) for each inherited port with a nonzero bit vector. Every simple edge has exactly one such owner. This uses at most K copies per node. Sets of original edges translate to finitely many monadic node sets. The original vertices remain exactly the tree nodes.

The following conditions are MSO on the finite ranked tree:

(i) the shape is the first-child/next-sibling encoding of a single ordered rooted tree, with valid port bounds and injective maps;

(ii) the decoded structure satisfies φ, and its decoded $<$ is a strict linear order;

(iii) inherited ports and siblings have the prescribed decoded order;

(iv) the inherited ports are *exactly* the set in (1), rather than an arbitrary bag superset;

(v) the encoded parent relation agrees with P_{can} on the decoded structure:

$$\forall u \, \forall v \quad [\mathrm{parent}_F(u, v) \iff P_{\mathrm{can}}^{\mathrm{dec}}(u, v)].$$

MSO backwards translation through the fixed-copy incidence decoding gives the last formula [10, Chapter 7, Theorem 7.10]. Original-vertex element variables remain node variables; edge-set variables become tuples of node-set variables, one per possible edge-owner copy. This is a translation with the displayed free variables u, v retained. Both sides compare relations on the *same node universe*; there is no need to express the existence of an isomorphism between unrelated structures.

By the MSO characterization of regular finite-tree languages [17, 11], these conditions define a regular language L. Every allowed structure has an accepted encoding. Conversely, a tree in L decodes an allowed structure and must use its canonical parent relation. Exact bags, port order, parent-port maps, edge ownership, and sibling order then force the unique encoding already described. Isomorphic decoded structures have isomorphic ordered labeled encodings. This proves the desired bijection, with one node per original vertex.

5.5 The polynomial system

Determinize a bottom-up automaton for L. For each state q, let $Y_q(t)$ count its accepted rooted trees by nodes. Then

$$Y_q(t) = t \sum_{a(q_1,\ldots,q_d)\to q} \prod_{j=1}^{d} Y_{q_j}(t), \qquad d \in \{0, 1, 2\}. \tag{2}$$

Different labels or ordered child tuples describe different trees; determinism assigns one state run to each tree. The system is finite and proper, with all $Y_q(0) = 0$. The Jacobian of its left side minus right side is the identity at $t = 0$, so its distinguished formal solution has algebraic components; this is the standard algebraicity of proper polynomial systems [12, Chapter VII]. Summing over accepting root states gives the desired algebraic series. This completes the proof of Theorem 5.1.

6 Application to generalized stacks

Fix positive r, s. Use the finite vocabulary consisting of the two event colors, directed successor, and symmetric matching. All binary relations are loopless. The underlying simple graph is the one in Theorem 4.1.

The diagram axioms of Lemma 2.4 are MSO. One may specify a first vertex with no predecessor, at most one successor per vertex, exactly one predecessor for every other vertex, and reachability of every vertex from the first. These conditions force a single directed path, excluding cycles and branching. Directed reachability itself is MSO: v is reachable from u if every successor-closed set containing u contains v. Its strict form gives the parameter-free time order.

Insertion and removal colors partition the vertices. A symmetric perfect matching joins opposite colors, and each insertion precedes its partner. The lazy-boundary condition is

$$I(u) \wedge R(v) \wedge S(u, v) \quad \Longrightarrow \quad M(u, v).$$

The perfect-matching axiom excludes unmatched removals and insertions; the displayed implication imposes the additional lazy boundary rule.

Identify tokens with insertion vertices and write $b(u)$ for the unique matching removal. For a monadic subset U of insertion vertices, the two endpoint sets are defined by

$$A_U(v) = \{u \in U : u < v < b(u)\}, \qquad B_U(v) = \{u \in U : u < b(v) < b(u)\}.$$

For fixed r, s, the inequalities $|A_U(v)| < r$ and $|B_U(v)| < s$ are MSO formulas with a fixed number of element witnesses: respectively, they forbid r or s pairwise distinct elements satisfying the displayed conditions. Thus

$$\forall U \subseteq I \quad [U \neq \varnothing \implies \exists v \in U \ (|A_U(v)| < r \ \wedge \ |B_U(v)| < s)] \tag{3}$$

is an MSO sentence. Proposition 3.3 proves that it characterizes physical feasibility exactly. It does not quantify a binary stack order or count its possible choices.

Every nonempty feasible diagram is connected, has its definable time order, and has treewidth at most $W = 12 \max(r, s) - 1$. Theorem 5.1 therefore gives an algebraic series

$$H(t) = \sum_{n \geq 1} g_{r,s}(n) t^{2n}.$$

The weight is $2n$ because each token has one insertion and one removal event; finite-copy incidence elements and intermediate decomposition nodes contribute no weight. It follows that

$$F_{r,s}(x) = 1 + H(\sqrt{x})$$

is algebraic over $\mathbb{Q}(x)$. Explicitly, if $p(t, H(t)) = 0$ for a nonzero $p \in \mathbb{Q}[t, y]$, then $p(t, y)p(-t, y)$ is a nonzero polynomial in t^2, y. Write it as $R(t^2, y)$. The even support of H gives an ordinary series $H(\sqrt{x})$ and $R(x, H(\sqrt{x})) = 0$, so $R(x, F_{r,s}(x) - 1) = 0$. The separate constant term counts the empty permutation once. Lemma 2.1 transfers the enumeration to sortable inputs, proving Theorem 1.1.

7 The explicit (3,3) equation

We now turn to Theorem 1.2. This result has an independent concrete proof; no large logical automaton from Theorem 5.1 is constructed. Its computer-assisted portions are exact identities and all-length recurrences, with a formal endpoint discussed in Section 8.

7.1 The selected scalar branch

Let

$$C_8 = 1 + x + 2x^2 + 6x^3 + 24x^4 + 120x^5 + 720x^6 + 5004x^7 + 38844x^8.$$

Exact substitution into the polynomial of Appendix A gives

$$Q(x, C_8 + x^9 z) = x^{20} R(x, z), \quad R(0, z) = -16(z - 325692)(z - 325656). \tag{4}$$

The quotient R is a polynomial, and $R_z(0, 325692) = -576$. The ordinary formal implicit-function theorem produces exactly one $z \in \mathbb{Q}[[x]]$ with $z(0) = 325692$ and $R(x, z) = 0$. Thus the displayed prefix uniquely selects a scalar branch. The weaker condition $G(0) = 1$ is insufficient. The identities in (4) are also checked by the small supplementary manuscript checker.

7.2 Why the scalar branch counts distinct permutations

Here we summarize the exact finite counting certificate used for identification. The complete tables, rational functions, division quotients, and combinatorial proofs are part of the supplements identified in Appendix B.

At an output boundary, the unique lazy code uses N_b for a new record obtained by reading b fresh labels, and H_j for the jth smallest held label. Different ranks remain different code letters even when their possible successor physical-stack sets coincide. A code determines its output permutation uniquely.

The concrete determinization represents the set of possible physical stacks in maximum-insertion coordinates: the coordinate of a label is the number of smaller labels above it. For top-three insertion the coordinates are bounded by 2, except for the forced initial bounds 0 and 1. An exact decorated factor language represents unions of successor orders, by the normal-form theorem and its suffix-image formulas in `docs/three-three-enumeration/decorated-normal-form.md`, Sections 1–3; Appendix B, supplement S3, identifies its proof supplements. Two counting reductions then produce a signed three-letter completion recurrence. One replaces a valid run ZO^a by Z^{a+1} through a relabelling of old tokens. The other replaces a clause C_m, behind a nonempty prefix, by

$$B^m - (B - Z)^m.$$

These restrictions matter: arbitrary isolated O factors cannot be erased, and the clause identity is justified by an output-language intersection theorem, not by assuming that unions of physical histories commute with intersections of output sets. The combinatorial proofs are included in the counting-reduction supplements.

The signed held-deletion operators admit a four-channel suffix transducer. Merging the scalar E channel reduces the suffix-channel representation from 24 to 19 coordinates; an exact intertwining quotient then reduces it from 19 to 9. A six-row invariant graph annihilates every word residual, provided seven scalar equations hold. Transition, initial-row, terminal, and exceptional boundary

identities are checked as rational identities, not on a finite inventory of words. In particular, the last residual column and the empty and height-one boundaries are included.

In the notation of the finite-system supplement, the seven equations and output are

$$(I_6 - xT)(I_6 + K)v - xv = 0, \qquad (\alpha - e_0)Kv = 0, \qquad v_0 = 1, \tag{5}$$

$$G = 1 + x + \frac{2x^2}{1 - xv_2}. \tag{6}$$

The first equality has six components. The matrices T, K, row α, and normalized column v are explicit rational functions of seven parameters (g, i, p, q, s, c, o). Here g is an auxiliary matrix parameter, and s is a parameter name local to this certificate, not a varying machine width. The normalized column is obtained from a five-by-five linear system whose determinant at the selected basepoint is 4.

Five explicit rational functions of (x, y) recover auxiliary coordinates (a, t, u, i, q) from the scalar root in (4). Their denominators have the following orders and nonzero leading coefficients on that root:

coordinate	a	t	u	i	q
order	10	12	10	12	10
leading coefficient	-186624	2304	-1036800	288	567787968

The numerator orders agree, giving constants $(2, 1/4, 1/6, 8, 1/18)$. These are regular series after common powers of x are cancelled; the original denominators are not incorrectly treated as units. Recover

$$g = \frac{u}{qt}, \qquad s = \frac{t}{q}, \qquad o = i - a, \qquad p = -iq,$$

and obtain c from its unit linear equation, whose coefficient has constant 6. The recovered tuple is

$$(g, i, p, q, s, c, o)(0) = (12, 8, -4/9, 1/18, 9/2, 2, 6).$$

All remaining chart denominators are units at this tuple.

The substitution into five reversibly recombined equations from (5), with denominators cleared exactly, is divisible by $Q(x, y)$. The supplied quotients establish all five equalities. A sixth exact divisibility establishes (6) with $G = y$. Undoing the triangular recombination and the unit c elimination gives the six components of the first equality in (5). The seventh, empty-boundary equation $(\alpha - e_0)Kv = 0$, vanishes identically after the already imposed substitution $p = -iq$. Thus all seven equations hold. Thus the scalar branch supplies a regular solution of the *complete* finite counting representation, with the prescribed output.

Finally, the completion recurrence has coefficientwise uniqueness. If a state has h held tokens and n unread inputs, a held deletion changes (n, h) to $(n, h - 1)$. A record reading $j + 1$ new inputs changes it to $(n - j - 1, h + j)$. Every summand therefore lowers $n + h$ by exactly one, and for each coefficient only finitely many choices occur. Strong induction on $n + h$ proves uniqueness, also for the signed recurrence. The actual distinct-output counts satisfy it, with the empty completion counted once. Hence the scalar solution is the true unrestricted generating function.

This argument does not require a capacity cutoff, coefficient extrapolation, assumed positivity, the older singular seven-variable Hensel argument, or the general logical algebraicity theorem. Finite branch jets are used only to certify specific valuations and unit denominators.

7.3 Exact algebraic degree

Specialize $x = 3$ and reduce modulo 17. In increasing powers of y, the resulting degree-12 polynomial has coefficient list

$$(5, 14, 9, 10, 4, 3, 14, 7, 14, 3, 14, 7, 5).$$

Let it be $f(y)$. Exact arithmetic in $\mathbb{F}_{17}[y]$ gives

$$y^{17^{12}} \equiv y \pmod{f}, \qquad \gcd(f, y^{17^6} - y) = \gcd(f, y^{17^4} - y) = 1.$$

Rabin's criterion [16] proves that f is irreducible. The finite certificate supplies explicit Bézout witnesses for the two gcds; the manuscript's independent small checker recomputes the modular powers and gcds directly.

The leading coefficient of Q in y is $972x^4$, and its constant coefficient in y is $-8100 - 11016x + 80028x^2$. They are coprime in $\mathbb{Q}[x]$. Thus Q has no nonconstant x-content. Its integer content is also 1, since its coefficients include -781 and 972 and $\gcd(781, 972) = 1$. A factorization over $\mathbb{Q}(x)$ would therefore, by Gauss's lemma, give an exact factorization into positive-y-degree factors over $\mathbb{Z}[x, y]$. Their leading coefficients cannot vanish under $x = 3$ modulo 17, since their product specializes to the nonzero leading coefficient of f. It would yield a nontrivial factorization of f, a contradiction. Therefore Q is irreducible over $\mathbb{Q}(x)$. Together with the counting identification, this proves that G has algebraic degree exactly 12. Its monic minimal polynomial is $Q/(972x^4)$.

8 Formal verification and reproducibility

The formal package deliberately has a narrower scope than the combined written results. Its final endpoint counts the duplicate-free finite set of outputs satisfying the literal predicate

$$\text{Run } 3\ 3\ [\,]\ (\text{List.range}\, n)\ \textit{output}\ [\,].$$

The input labels are $0, \ldots, n-1$; relabelling them to $1, \ldots, n$ changes no operation. Histories and alternative witnesses are not counted more than once. The stack has no fixed height bound and n is arbitrary.

In the namespace `StackSorting.Generalized.Decorated.SignedCompletion`, the endpoint `literal_G_counts` takes only a natural-number index and identifies every coefficient of the constructed scalar branch with this cardinality. The theorem `actual_counts_literalQ` takes no arguments and states the literal scalar identity coefficientwise for the actual counting series. Series multiplication is finite Cauchy convolution.

The accompanying source-only certificate contains 1,613 modules for Lean 4.34.1. The endpoint axiom reports contain only `propext`, `Classical.choice`, and `Quot.sound`. The source manifest identifies every module by its exact hash and records the dependency order. Four unchanged modules retain explicitly documented warning exceptions; all other modules are compiled with warnings treated as errors. The reproduction script uses an isolated build and does not reuse supplied project objects. Source-integrity checks are separate from compiler runs.

The endpoint establishes actual generated-output counts. It is not directly stated as a theorem about a separate sortable-input predicate; Lemma 2.1 gives the written equinumerous interpretation. It does not certify irreducibility, minimal algebraic degree, Theorem 1.1, the external MSO theorems, or analytic asymptotics. These exclusions are substantive.

As finite diagnostics only, an independent literal physical search gave

$$1, 1, 2, 6, 24, 120, 720, 5004, 38844$$

through size eight. At size two there are five complete histories but two distinct outputs. At size seven, capacity three gives 486 outputs whereas unrestricted height gives 5004. These controls illustrate why both the distinct-output convention and the absence of a capacity cutoff matter; they are not the proof of the all-length theorem.

9 Companion results and further questions

The present paper's general proof is independent of the project's earlier direct structural derivation for $(2, 2)$. That companion addresses the structural enumeration task in Vatter's separate Problem 5.11 [18]. That baseline and the separate physical-capacity manuscript are preserved as companion works. A further classification manuscript treats finite versus infinite bases. Its formal scope is stated separately.

For the unrestricted $(3, 3)$ class, the global basis-width bound $2\max(r, s) + 1$ combines with an exact 28-state lazy-code grammar to describe the entire ordinary basis. Its basis generating function is

$$B_{3,3}(x) = \frac{36x^7(1 + 3x + 2x^2 + 3x^3 + x^5 - x^6)}{1 - 3x^3 - x^5}.$$

This counts minimal forbidden permutations, not the whole class counted by G. The complete-basis proof and grammar are supplementary companion material, not consequences of the degree-12 scalar relation. The sorting-oriented basis is obtained by inversion.

Several concrete questions remain. The general canonical-tree construction gives no practical state bound suited to computation. It would be useful to obtain small explicit polynomial systems at further widths, sharper geometric width bounds, and singularity analyses of the concrete series. Historical comparison of the generic ordered counting lemma deserves further attention independently of the stack application. None of these open tasks is claimed to be settled here.

Data and code availability

The companion source distribution contains the manuscript sources, proof supplements, exact polynomial data, verification scripts, and Lean certificates. Paths in Appendix B are relative to that distribution. The papers and original figures are licensed under Creative Commons Attribution 4.0 International; original software and Lean sources are licensed under the MIT License. Third-party material retains its own license and attribution.

A The explicit scalar polynomial

Write $Q(x,y)=\sum_{j=0}^{6}x^jA_j(y)$, where

$$\begin{aligned}
A_0(y) &= -(y-1)^4(11y-18)(71y^3-414y^2+783y-450),\\
A_1(y) &= -2(y-1)^3(744y^6-6811y^5+25869y^4\\
&\qquad -48798y^3+41724y^2-7398y-5508),\\
A_2(y) &= (y-1)^2(5103y^8-40638y^7+119399y^6-150604y^5\\
&\qquad +94714y^4-174168y^3+373266y^2-307764y+80028),\\
A_3(y) &= -4y(y-1)(999y^9-7497y^8+17661y^7-16536y^6\\
&\qquad +43581y^5-155839y^4+204517y^3-60318y^2\\
&\qquad -57006y+30303),\\
A_4(y) &= y^2(972y^{10}-6912y^9+8604y^8-23076y^7+251778y^6\\
&\qquad -671588y^5+524094y^4+235008y^3-434593y^2\\
&\qquad +67968y+47583),\\
A_5(y) &= 2y^3(972y^8+10260y^7-61902y^6+77520y^5\\
&\qquad +38718y^4-97642y^3+4688y^2+24047y+2610),\\
A_6(y) &= -y^4(6y^2-10y-5)\\
&\qquad \cdot(864y^5-1242y^4-1458y^3+1563y^2+716y-20).
\end{aligned}$$

Expansion gives exactly the 69 coefficients in the accompanying JSON file. The total constant term is -8100, and the coefficient of y^{12} is $972x^4$, so the relation is nonzero. The degree-9 branch coefficient in Theorem 1.2 is necessary to distinguish the selected branch of (4).

B The exact proof and data supplements

The source distribution contains the following supplements. The top-level source manifest gives their byte hashes, and `REPRODUCE.md` gives the supported verification commands.

S1. General algebraicity. The current proof is Section 5, including its explicit introduction-preorder construction. The supporting endpoint formula is in `research/ordered-mso-algebraicity/endpoint-class-formula.md`.

S2. Stack-specific inputs. Lazy normalization is in `docs/composition-normal-form.md`, Section 2; endpoint peeling in `docs/generalized-basis-width.md`, Sections 1–3; and the geometric argument in `research/geometric-width/geometric-width-route-v2.md`.

S3. Signed counting and all-word representation. The normal-form theorem and suffix-image formulas are in `docs/three-three-enumeration/decorated-normal-form.md`, Sections 1–3. Counting reductions, the complete residual graph and the explicit matrix construction are in `research/full-class-equation/author/`; `explicit-seven-equation-theorem.md` gives the matrix and linear-system recipes in (5).

S4. Direct scalar equation and minimality. `research/direct-counting-equation/` contains the equation, branch, rational reconstruction, divisibility witnesses, reversible lift and finite-field irreducibility certificate. The source polynomial is `author/direct-G33-equation.json`.

S5. Formal actual-count equation. `formal/actual-scalar-equation/` contains the transparent 1,613-module source closure, exact hashes, warning policy and source-only replay for Lean 4.34.1. Compiler binaries and compiled proof objects are not included.

S6. Complete ordinary $(3,3)$ basis. `docs/three-three-complete-basis.md` gives the 28-state grammar and rational basis series. Its width and bounded-grammar dependencies are separate from the main counting series and lie outside the actual-count-to-Q Lean endpoint.

The manuscript's local checker verifies only the displayed polynomial, branch blowup, finite-field tests, and agreement with the frozen JSON. It does not rerun the large formalization or independently reprove the MSO bridge. Build and verification commands are in the accompanying README.

References

[1] Michael H. Albert and Michael D. Atkinson. Sorting with a forklift. *The Electronic Journal of Combinatorics*, 9(2):R9, 2003. URL https://arxiv.org/abs/cs/0209016.

[2] Michael H. Albert, Cheyne Homberger, Jay Pantone, Nathaniel Shar, and Vincent Vatter. Generating permutations with restricted containers. *Journal of Combinatorial Theory, Series A*, 157:205–232, 2018. URL https://arxiv.org/abs/1510.00269.

[3] M. D. Atkinson. Generalized stack permutations. *Combinatorics, Probability and Computing*, 7 (3):239–246, 1998. doi: 10.1017/S096354839800354X.

[4] Hans L. Bodlaender. A partial k-arboretum of graphs with bounded treewidth. *Theoretical Computer Science*, 209:1–45, 1998. doi: 10.1016/S0304-3975(97)00228-4.

[5] Mikołaj Bojańczyk. Languages recognised by finite semigroups, and their generalisations to objects such as trees and graphs, with an emphasis on definability in monadic second-order logic. Lecture notes, arXiv:2008.11635v1, 2020. URL https://arxiv.org/abs/2008.11635v1. Definitions 6.20–6.21 and Theorem 6.22; version-pinned lecture notes.

[6] Mikołaj Bojańczyk and Michał Pilipczuk. Definability equals recognizability for graphs of bounded treewidth. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 407–416, 2016. doi: 10.1145/2933575.2934508. URL https://arxiv.org/abs/1605.03045.

[7] Mikołaj Bojańczyk and Michał Pilipczuk. Optimizing tree decompositions in MSO. *Logical Methods in Computer Science*, 18(1):26:1–26:36, 2022. doi: 10.46298/LMCS-18(1:26)2022. URL https://arxiv.org/abs/1701.06937.

[8] Samuel Braunfeld. Decidability in geometric grid classes of permutations. *Proceedings of the American Mathematical Society*, 153(3):987–1000, 2025. doi: 10.1090/proc/17083. URL https://arxiv.org/abs/2308.04201v2.

[9] Bruno Courcelle. The monadic second-order logic of graphs V: on closing the gap between definability and recognizability. *Theoretical Computer Science*, 80(2):153–202, 1991. doi: 10.1016/0304-3975(91)90387-H.

[10] Bruno Courcelle and Joost Engelfriet. *Graph Structure and Monadic Second-Order Logic: A Language-Theoretic Approach*, volume 138 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 2012. doi: 10.1017/CBO9780511977619.

[11] John Doner. Tree acceptors and some of their applications. *Journal of Computer and System Sciences*, 4(5):406–451, 1970. doi: 10.1016/S0022-0000(70)80041-1.

[12] Philippe Flajolet and Robert Sedgewick. *Analytic Combinatorics*. Cambridge University Press, 2009. URL https://ac.cs.princeton.edu/home/.

[13] Radu Iosif and Florian Zuleger. Characterizations of monadic second order definable context-free sets of graphs. *Logical Methods in Computer Science*, 22(1):22:1–22:30, 2026. URL https://arxiv.org/abs/2310.04764.

[14] Vít Jelínek and Michal Opler. Monadic second-order logic of permutations. arXiv:2511.02386v1, 2025. URL https://arxiv.org/abs/2511.02386v1.

[15] Ioannis Katsikarelis. Computing bounded-width tree and branch decompositions of k-outerplanar graphs. arXiv:1301.5896, 2013. URL https://arxiv.org/abs/1301.5896.

[16] Michael O. Rabin. Probabilistic algorithms in finite fields. *SIAM Journal on Computing*, 9(2): 273–280, 1980. doi: 10.1137/0209024.

[17] James W. Thatcher and Jesse B. Wright. Generalized finite automata theory with an application to a decision problem of second-order logic. *Mathematical Systems Theory*, 2:57–81, 1968. doi: 10.1007/BF01691346.

[18] Vincent Vatter. An assortment of problems in permutation patterns: unimodality, equivalence, derangements, and sorting. arXiv:2602.16355v2, 2026. URL https://arxiv.org/abs/2602.16355v2. Section 5, especially Question 5.12.